

Data Quality and Zipf's Law

Chris Nuhn

Advisor: Dr. Valerie Sessions,

November 29, 2010

TABLE OF CONTENTS

SENIOR PROJECT	
INTRODUCTION	3
<i>Project Background and Purpose</i>	3
<i>Project Summary</i>	3
<i>Project Timeline</i>	3
RESEARCH	
<i>Central Limit Theorem</i>	4
<i>Normal Distributions</i>	4
<i>Power-rank and Zipf's Law</i>	5
<i>Determining Outliers</i>	5
<i>Data Creation</i>	6
<i>Data Analysis</i>	8
CODE BREAKDOWN	
<i>Creating Corrupt Data</i>	9
<i>Zipf's Law and Correlation Coefficient</i>	10
PROBLEMS ENCOUNTERED	
<i>Format</i>	11
<i>Random Number Generator</i>	11
<i>Log Bases</i>	11
REFERENCES.....	
APPENDIXES.....	
A. COMPLETE DATA	12
B. PROJECT CODE	17
C. 2010 ICIQ RESEARCH PAPER.....	22
D. 2010 SPRING SYMPOSIUM PRESENTATION	36
E. SENIOR DEFENSE PRESENTATION	39

Introduction

Project Background and Purpose

This program is designed to test if there are other ways to find errors in data sets. The traditional way to do this is to plot the data on a graph and see if it is normally distributed. If it is, then this allows the user to use a variety of formulas such as mean, median, and mode to see if the data is accurate. However, not all data fits this distribution and when assumed to be normally distributed, errors can be missed. This project is geared toward finding errors in data sets that fit power law distributions. A specific version of a power law distribution is the Zipfian distribution. In this type of distribution, a set of data is reordered in descending order and then the log of the ranks is plotted versus a log of the values. If this plot is linear, it follows Zipf's Law.

Project Summary

This program outputs how well a certain set of data fits to Zipf's law based on the correlation to a line. The user is able to give this program a .txt file with data in numerical form and outputs the correlation, as well as being able to create a .xls file that can easily be graphed. As part of the research involved with this project, the program is able to create errors in the data in order to see if those outliers can be spotted. The program is able to randomly generate corrupt data two separate ways, allowing for more accurate comparisons to be made.

Project Timeline

January 2010	-	Research Zipf's Law
January 2010	-	Find data sets that fit Zipf's Law
February 2010	-	Find methods to determine outliers
February 2010	-	Code research script
March 2010	-	Analysis data
March 2010	-	Spring Symposium
April – June 2010	-	Review Research
July 2010	-	Submit paper to ICIQ

RESEARCH

Research

Central Limit Theorem

To start off in our research, we explored the reason the Normal distribution is so commonly used. This is because of the Central Limit Theorem, which states that any set of data with a finite mean μ and variance σ^2 , tends to be normally distributed as the sample size increases. The key-word in that sentence is “tends.” This is because we have explored data that would be assumed to be normal, when in-fact a different distribution would fit much better.

Normal Distributions

The Normal distribution, more commonly known as a bell-curve, incorporates a finite

mean and variance as stated above. The distribution follows the format: $F(x) = \frac{e^{-\frac{(x-\mu)^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}}$, where x is the variant with mean μ and variance σ^2 . Once a data set is found to be normal, certain properties are unlocked for use. The distribution is symmetric, with equal amounts of data on either side of the center, the data rapidly decreases from the center and is less frequent at the extremes. Also, the area under the curve is proportional to the probability of that value or range of values.

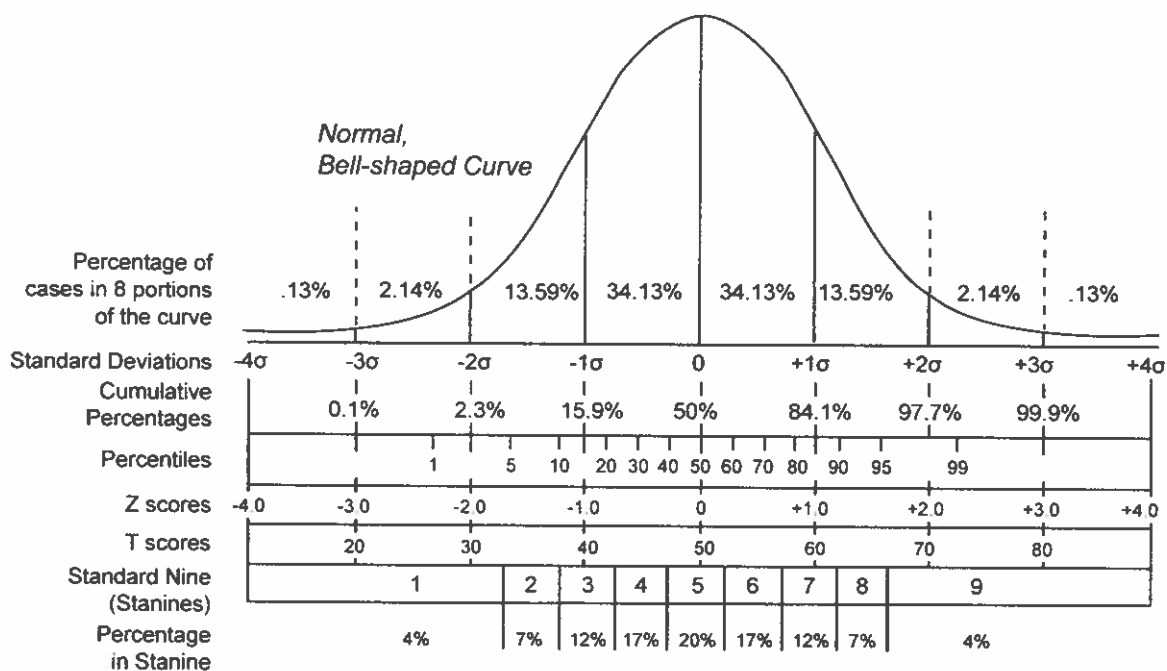


Figure 1: Normal Distribution

Power-rank and Zipf's Law

The Power-rank Law has been studied by many scientists, most famously Gauss, Pareto, and George Kingsley Zipf. Pareto explored the phenomenon that 80% of a nation's wealth is held by 20% of the population, hence his "80-20 theory." Upon this observation, he noticed that the values appear to be exponential. Building upon this, Zipf observed that if the values are ranked in descending order, the values are then inversely proportional to its new rank. There-in, rank 2 occurs $\frac{1}{2}$ rank 1 and rank 3, $\frac{1}{3}$ and so on. Therefore, when the log of the ranks is plotted against the log of the values, a linear plot is formed. Note this works for any log base, such as 10, 2, or e.

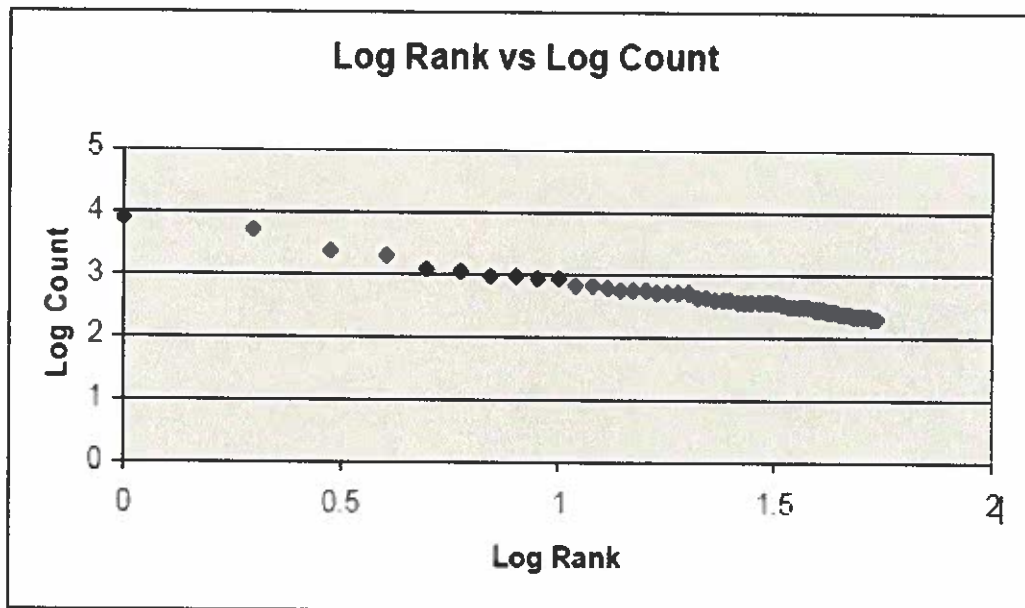


Figure 2: Log Rank v. Log Count for ALICE bot input.

Power-rank is normally seen in the format: $F(x) = ax^k + o(x^k)$ where a and k are constants with $o(x^k)$ is an asymptotically small function of x^k . However, once the log-log graph is plotted, a correlation coefficient is needed to determine the proximity to a line. This coefficient, r , is given as a range between $[-1,1]$ such that the closer to 1 or -1, the closer to a line. The negative determines if the slope is increasing or decreasing.

Determining outliers

An outlier is an element that appears to deviate markedly from other members of the sample in which it occurs. This definition is given by Frank E. Grubbs, also the creator of the Grubbs' Test for outliers. His test only works for normal distributions and states that a data point is an outlier if it is beyond: $\frac{Y - Y_{\min}}{s}, \frac{Y - Y_{\max}}{s}$ where Y is the sample mean and s is the standard deviation of the set. This seems straight forward, and it is, however there is no such test for a Zipfian distribution. To determine if a data point is an

outlier, we must look at the entire data's correlation to a line when plotted as log rank v. log value. For this project we determined that a data set followed Zipf's law if the correlation coefficient was within the 90th percentile, i.e. between [0.9,1]. Therefore, if a data point took the set out of this range, it was considered an outlier.

Data Creation

One of the initial steps in this project was to find a set of data that would normally be considered "normal" and find and error in determining outliers. We found one such example with the data from the ALICE bot, created by Dr. Richard S. Wallace of the ALICE Artificial Intelligence foundation. Alice is a computer chat bot that gives responses to a input given to it. The data we researched is based on the most common inputs that she receives. The portion of data used is given in Table 1 below and Figure 3 represents the distribution of the set.

At first glance, one would consider this normal and go about calculating it's mean, median, and mode trying to describe the uniqueness of this graph. However, consider point #21, value 846. If this point were to suddenly increase by a factor of 2, becoming 1692, it would still fit inside the data range without much problem and would not be singled out as an outlier. But when this data is plotted using the Zipfian distribution, as seen in Figure 2 and represented by Table 2, an increase by a factor of 2 for the same value would skew the line dramatically and be flagged as an outlier.

Number	Count	Number	Count
1	214	28	1145
2	227	29	946
3	233	30	840
4	261	31	645
5	277	32	553
6	288	33	544
7	297	34	488
8	303	35	486
9	309	36	414
10	352	37	403
11	366	38	381
12	368	39	375
13	374	40	353
14	403	41	349
15	416	42	340
16	516	43	300
17	525	44	296
18	531	45	286
19	584	46	262
20	663	47	251
21	846	48	239
22	934	49	224
23	1101	50	223
24	2268	51	218
25	2006	52	217
26	8024	53	203
27	5184	54	201

Table 1: ALICE bot input.

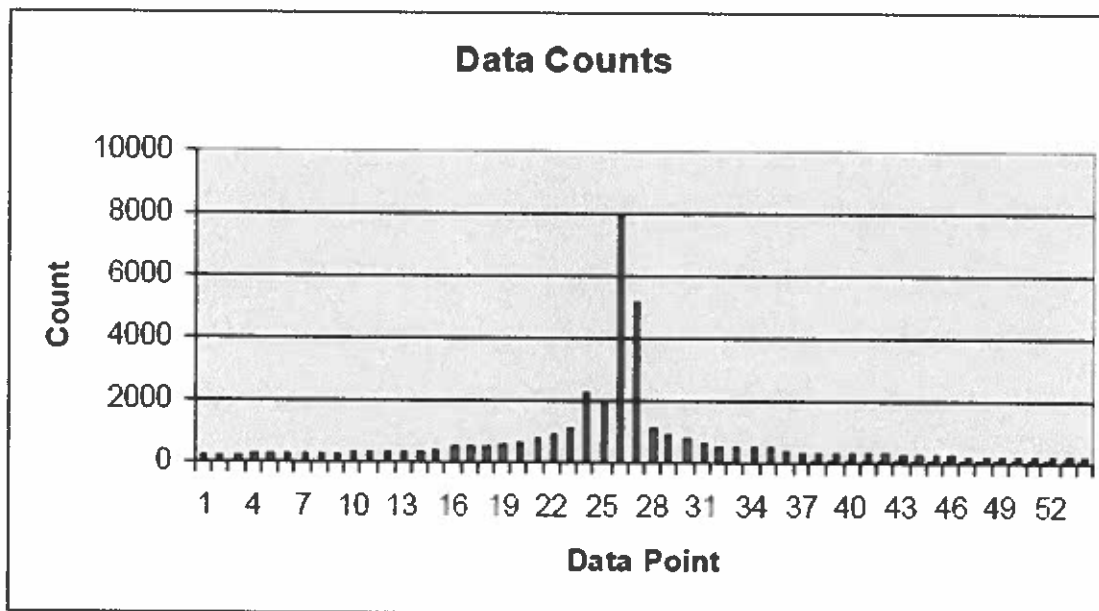


Figure 3: Normal Distribution of ALICE bot

Rank	Count	Rank	Count
1	8024	28	368
2	5184	29	366
3	2268	30	353
4	2006	31	352
5	1145	32	349
6	1101	33	340
7	946	34	309
8	934	35	303
9	846	36	300
10	840	37	297
11	663	38	296
12	645	39	288
13	584	40	286
14	553	41	277
15	544	42	262
16	531	43	261
17	525	44	251
18	516	45	239
19	488	46	233
20	486	47	227
21	416	48	224
22	414	49	223
23	403	50	218
24	403	51	217
25	381	52	214
26	375	53	203
27	374	54	201

Table 2: Rank v. Count for ALICE bot

Following this observation we now took it a step further and examined a multitude of other data sets and what kind of errors it would take for it to be flagged as corrupt or containing outliers. Some of the data we tested were:

1. Word Counts – for the novel Moby Dick by Herman Melville.
2. Solar Flares – the count of the peak intensity between 1980 and 1989.
3. Names – the frequency of family surnames in the 1990 U.S. census.
4. Citations – Author citations in June 1997 by authors published in 1981
5. Religious Affiliations – Religious affiliations in the U.S.

To corrupt the data, we changed a percentage of the values and exchanged them with randomly generated values. We incorporated two different methods for creating the randomly generated values. The first method randomly selected a value between 0 and 10% higher than the maximum value. The second chose a random number between 0 and 10% higher than the maximum value, and once again selected random numbers between 0 and that generated number. To compare the data, we ran the authentic set through the Zipf's law test as well and called it the "Gold Standard." To view all the data and results, reference Appendix A.

Data Analysis

After creating this data, there are two aspects that should be explored, the slope and correlation coefficient. First to be noted is the slope. As can be seen from the tables in Appendix A, the slopes vary but then tend back toward the Gold Standard with the exception of Data set E. Thus making it almost useless to tell outliers and how corrupt the data is. This may be a result of the corrupt data forming its own new line as the data approaches being mostly corrupt with only a few real data points.

Second we examine the correlation coefficients. There is actually a trend here where data deviates from the gold standard as can be expected. After being between 6 – 8% corrupt the data goes beyond the acceptable percentile we created to be called Zipfian. Furthermore, after 12%, the data tends to get outside the 80 percentile which shows that our methods for identifying outliers looks promising.

Also, note that the difference between the two methods of corrupting data were very similar and yielded very similar results.

CODE BREAKDOWN

Code Breakdown

Creating Corrupt Data

```
$rank = 0;
foreach (<IN>)
{
    chomp $_;
    next unless $_ =~ m/\t*(\d+)\t*/;
    $y[$rank] = $_;
    $rank++;
}
```

In this excerpt, the program is taking the .txt file given and breaking it down into an array. The loop chops off the return character, /n, and uses a pattern matcher to extract the numerical values. After this is done, it inserts the number into an array ranked with its position in the file, which may not be in order.

```
@y = sort {$b <=> $a} @y;
foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.
30,.35,.40,.45,.50)
{
```

This part of the code then sorts the array such that all the values are in the array in rank, or descending order. Once this is complete, it then runs a loop 22 times with the percentages to distort the original data.

```
$amount = int($rank * $_);
$stopvalue = int($y[0] * 1.1);
for($i = 0; $i < $amount; $i++)
{
    $new[$i] = rand($stopvalue);
}
foreach (@new)
{
    $y[rand($rank)] = int($_);
}
open( OUT, ">$_"."$file");
foreach (@y)
{
    print OUT $_, "\n";
}
```

Finally, the program assess how many times it must change data and runs nested loops to change the data in random spots with random data, in this case a number between 0 and 10% higher than the maximum value. After completion, it then writes the new array into a uniquely named .txt file with the new data.

Zipf's Law and Correlation Coefficient

```
#####
#          ARGS          #
#####
# 1. String      File      #
# 2. Boolean     Create .xls  #
# 3. Boolean     Print      #
#####
if ($#ARGV >= 2) {$print = $ARGV[2];}
if ($#ARGV >= 1) {$xls = $ARGV[1];}
if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
if ($#ARGV < 0) {$error = "file not given\n";}
```

It is important to know what the user wants to do with the data and to not waste unnecessary time working on something that the user isn't going to use. Therefore, here I have setup the command line arguments that the program accepts. The first is the file that the user is testing, which is required, while the creating of an excel .xls and print the file is second and third respectively. Do note that the booleans will default to false if not given.

```
sub correlation
{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sdx = sd($ref_x);
    my $sdy = sd($ref_y);
    my $xbar = mean($ref_x);
    my $ybar = mean($ref_y);
    my $sum = 0;
    foreach (0..($n - 1))
    {
        $sum += ((@{$ref_x}[$_] - $xbar)*(@{$ref_y}[$_] - $ybar));
    }
    my $corr = ($sum / (($n) * $sdx * $sdy));
    if($print) {print "Correlation:\nr: $corr\n";}
    return $corr;
}
```

This is an excerpt of my correlation coefficient subroutine, although not the longest, defiantly the most complicated. This subroutine takes two arrays by reference and a boolean and must create standard deviations for the two arrays as well as finding their

means. The equation being expressed by this subroutine is:
$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n-1)S_x S_y}$$

where the x's and y's are coming from the arrays, $\bar{x}$ and $\bar{y}$ are the means of the arrays, and S_x and S_y are the standard deviations.

PROBLEMS ENCOUNTERED

Problems Encountered

Format

On my first attempt to take in a .txt file, I was allowing the user to have the names of the fields, which corresponded to what the data represented. I then tried to pattern match out the numerical data and save the text elsewhere to reference. This became a problem because each set of data was formatted differently and depending if it came from a table, it may have had random tabs and spaces. After failing to find an efficient way to the data straight I changed the requirements for the .txt file to only have the numerical data with only return characters at the end of every line.

Random Number Generator

Upon research into how perl generates the random numbers I discovered that there was a problem with the seeds in versions earlier than v5.2. The version I compiled the research data was under v5.10.1. It was estimated that the previous versions only selected seeds from among 33,000 choices.

Log Bases

When creating the initial .xls files, I noticed that the numbers for my log(ranks) and log(values) differed from what I had calculated using excel. After research into perl's method for taking logs, I discovered that perl uses the default base 2 for most computing perposes instead of the standard log base 10 for mathematics. This however was an easy

fix by using the change in base formula: $\log_B(x) = \frac{\log_A(x)}{\log_A(B)}$. This has no effect on the

outcome of Zipf's Law, because base does not matter, however I choose to use the mathematical standard of base 10.

Works Cited

- Adamic, Lada A.. "Zipf, Power-laws, and Pareto - a ranking tutorial." 28 Nov. 2010.
Information Dynamics Lab, HP Labs.
<<http://www.hpl.hp.com/research/idl/papers/ranking/ranking.html>>.
- "Grubbs' Test for Outliers." 28 Nov. 2010. Nation Institute of Standards and Technology.
<<http://www.itl.nist.gov/div898/handbook/eda/section3/eda35h1.htm>>.
- Simon, Steve. "Stats: What is a normal distribution?." 28 Nov. 2010. children's Mercy
Hostpitals and Clinics. 11 Oct. 2002
<http://www.cmh.edu/stats/definitions/norm_dist.htm>
- "The Central Limit Theorem." California State University San Bernardino. 28 Nov. 2010
<<http://www.math.csusb.edu/faculty/stanton/probstat/clt.html>>.

APPENDIX A.

Complete Data

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-1.113275	-0.988579	0
1%	-1.330470	-0.977787	0.217195
2%	-1.658394	-0.958271	0.545119
3%	-2.025777	-0.946614	0.912502
4%	-2.383607	-0.937757	1.270332
5%	-2.706081	-0.926703	1.592806
6%	-2.969563	-0.911636	1.856288
7%	-3.157777	-0.892204	2.044502
8%	-3.266302	-0.870329	2.153027
9%	-3.252679	-0.857065	2.139403
10%	-3.192771	-0.837566	2.079495
11%	-3.085507	-0.814163	1.972231
12%	-2.947045	-0.788848	1.833770
13%	-2.783619	-0.762256	1.670344
14%	-2.598766	-0.733374	1.485491
15%	-2.410874	-0.706243	1.297599
20%	-2.173447	-0.673616	1.060172
25%	-1.912633	-0.638245	0.799358
30%	-1.661293	-0.606023	0.548018
35%	-1.484651	-0.612337	0.371376
40%	-1.298752	-0.614250	0.185476
45%	-1.184591	-0.636806	0.071315
50%	-1.091816	-0.667539	-0.021460

Table 3: Data Set A

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-1.069828	-0.992570	0.000000
1%	-1.239464	-0.977897	-0.169636
2%	-1.497125	-0.960423	-0.427297
3%	-1.796500	-0.948668	-0.726672
4%	-2.099655	-0.939553	-1.029827
5%	-2.367906	-0.928075	-1.298078
6%	-2.580713	-0.913613	-1.510885
7%	-2.732337	-0.895342	-1.662509
8%	-2.823911	-0.872613	-1.754083
9%	-2.853433	-0.847017	-1.783605
10%	-2.825839	-0.819033	-1.756011
11%	-2.751598	-0.790224	-1.681770
12%	-2.636783	-0.760147	-1.566955
13%	-2.497380	-0.729522	-1.427552
14%	-2.340690	-0.700011	-1.270862
15%	-2.172721	-0.670777	-1.102893
20%	-1.955555	-0.641851	-0.885727
25%	-1.735631	-0.615828	-0.665803
30%	-1.514631	-0.587774	-0.444803
35%	-1.299938	-0.567309	-0.230110
40%	-1.106034	-0.552448	-0.036206
45%	-0.964522	-0.551619	0.105306
50%	-0.863613	-0.559030	0.206215

Table 4: Data Set B

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-0.852299	-0.994842	0.000000
1%	-0.920012	-0.997266	-0.067713
2%	-1.042925	-0.993554	-0.190626
3%	-1.168022	-0.987005	-0.315723
4%	-1.315955	-0.975075	-0.463656
5%	-1.447085	-0.962358	-0.594786
6%	-1.564527	-0.945028	-0.712228
7%	-1.640417	-0.926630	-0.788118
8%	-1.690200	-0.903462	-0.837901
9%	-1.700633	-0.879958	-0.848334
10%	-1.691585	-0.853686	-0.839286
11%	-1.655821	-0.832537	-0.803522
12%	-1.595099	-0.810317	-0.742800
13%	-1.524150	-0.785785	-0.671851
14%	-1.446234	-0.759144	-0.593935
15%	-1.368551	-0.738759	-0.516252
20%	-1.267139	-0.714443	-0.414840
25%	-1.151323	-0.689070	-0.299024
30%	-1.045189	-0.663526	-0.192890
35%	-0.927194	-0.638208	-0.074895
40%	-0.819479	-0.624336	0.032820
45%	-0.755504	-0.621493	0.096795
50%	-0.712343	-0.634528	0.139956

Table 5: Data Set C

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-2.497718	-0.987608	0.000000
1%	-2.678983	-0.992400	-0.181265
2%	-2.936413	-0.990630	-0.438695
3%	-3.289699	-0.981541	-0.791981
4%	-3.640914	-0.966808	-1.143196
5%	-3.961571	-0.945712	-1.463853
6%	-4.167570	-0.925201	-1.669852
7%	-4.306592	-0.902501	-1.808874
8%	-4.379137	-0.873514	-1.881419
9%	-4.370368	-0.839687	-1.872650
10%	-4.260689	-0.806561	-1.762971
11%	-4.073511	-0.766355	-1.575793
12%	-3.841621	-0.726180	-1.343903
13%	-3.599693	-0.692132	-1.101975
14%	-3.311202	-0.660768	-0.813484
15%	-2.988006	-0.621011	-0.490288
20%	-2.681185	-0.587464	-0.183467
25%	-2.341687	-0.548562	0.156031
30%	-1.927895	-0.509061	0.569823
35%	-1.602143	-0.480668	0.895575
40%	-1.346614	-0.467117	1.151104
45%	-1.111651	-0.466787	1.386067
50%	-1.023600	-0.502952	1.474118

Table 6: Data Set D

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-3.170003905	-0.939689	0.000000
1%	-3.170003905	-0.939689	0.000000
2%	-3.170003905	-0.939689	0.000000
3%	-3.170003905	-0.939689	0.000000
4%	-3.170003905	-0.939689	0.000000
5%	-3.196891387	-0.916498	-0.026887
6%	-3.166925523	-0.912175	0.003078
7%	-3.187790214	-0.905218	-0.017786
8%	-3.028894425	-0.884737	0.141109
9%	-3.025066111	-0.884299	0.144938
10%	-3.036999758	-0.872218	0.133004
11%	-2.80998711	-0.809960	0.360017
12%	-2.677691993	-0.761718	0.492312
13%	-2.667245636	-0.751060	0.502758
14%	-2.653013085	-0.740559	0.516991
15%	-2.634523768	-0.745369	0.535480
20%	-2.668712705	-0.774571	0.501291
25%	-2.089574955	-0.712225	1.080429
30%	-2.046534647	-0.714426	1.123469
35%	-1.84755318	-0.767803	1.322451
40%	-1.263565195	-0.753158	1.906439
45%	-1.015029791	-0.776767	2.154974
50%	-1.121950978	-0.712290	2.048053

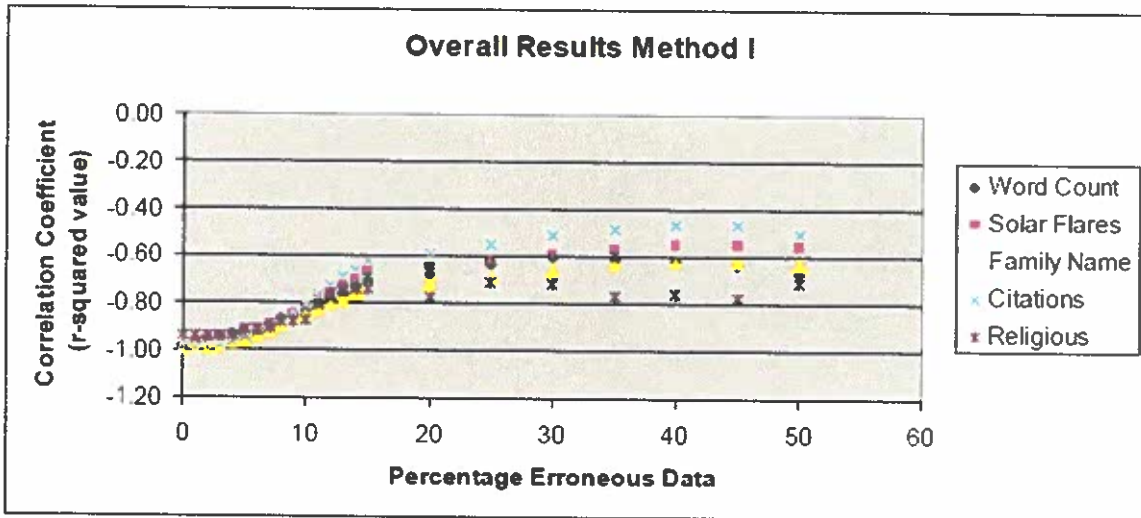
Table 7: Data Set E

Percent Corrupt	R - squared value Random Method I				
	Word Count	Solar Flares	Family Name	Citations	Religious
Gold Standard - 0%	-0.988579	-0.992570	-0.994842	-0.987608	-0.939689
1%	-0.977787	-0.977897	-0.997266	-0.992400	-0.939689
2%	-0.958271	-0.960423	-0.993554	-0.990630	-0.939689
3%	-0.946614	-0.948668	-0.987005	-0.981541	-0.939689
4%	-0.937757	-0.939553	-0.975075	-0.966808	-0.939689
5%	-0.926703	-0.928075	-0.962358	-0.945712	-0.916498
6%	-0.911636	-0.913613	-0.945028	-0.925201	-0.912175
7%	-0.892204	-0.895342	-0.926630	-0.902501	-0.905218
8%	-0.870329	-0.872613	-0.903462	-0.873514	-0.884737
9%	-0.857065	-0.847017	-0.879958	-0.839687	-0.884299
10%	-0.837566	-0.819033	-0.853686	-0.806561	-0.872218
11%	-0.814163	-0.790224	-0.832537	-0.766355	-0.809960
12%	-0.788848	-0.760147	-0.810317	-0.726180	-0.761718
13%	-0.762256	-0.729522	-0.785785	-0.692132	-0.751060
14%	-0.733374	-0.700011	-0.759144	-0.660768	-0.740559
15%	-0.706243	-0.670777	-0.738759	-0.621011	-0.745369
20%	-0.673616	-0.641851	-0.714443	-0.587464	-0.774571
25%	-0.638245	-0.615828	-0.689070	-0.548562	-0.712225
30%	-0.606023	-0.587774	-0.663526	-0.509061	-0.714426
35%	-0.612337	-0.567309	-0.638208	-0.480668	-0.767803
40%	-0.614250	-0.552448	-0.624336	-0.467117	-0.753158
45%	-0.636806	-0.551619	-0.621493	-0.466787	-0.776767
50%	-0.667539	-0.559030	-0.634528	-0.502952	-0.712290

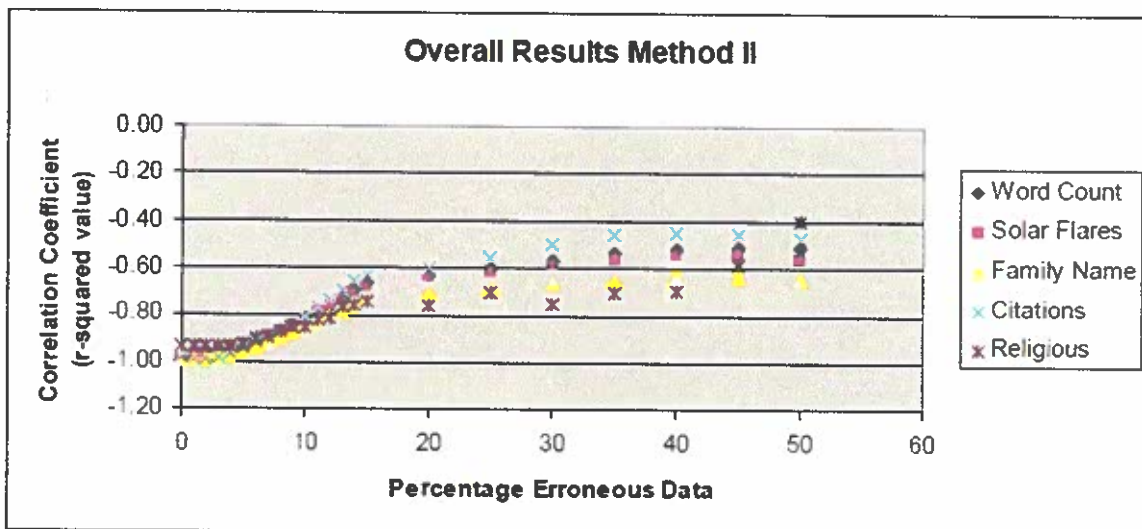
Table 8: Overall Data – Method I

Percent Corrupt	R - squared value Random Method II				
	Word Count	Solar Flares	Family Name	Citations	Religious
Gold Standard - 0%	-0.988579	-0.992570	-0.994842	-0.987608	-0.939689
1%	-0.978572	-0.981384	-0.995643	-0.992367	-0.939689
2%	-0.961307	-0.966405	-0.993760	-0.990391	-0.939689
3%	-0.950432	-0.954958	-0.986636	-0.981653	-0.939689
4%	-0.941061	-0.944805	-0.975702	-0.966876	-0.939689
5%	-0.929961	-0.933324	-0.961673	-0.947337	-0.925849
6%	-0.915121	-0.918234	-0.945889	-0.922518	-0.905163
7%	-0.896919	-0.899130	-0.927446	-0.899288	-0.893834
8%	-0.873743	-0.876048	-0.905224	-0.869428	-0.869612
9%	-0.847091	-0.852133	-0.881373	-0.834736	-0.856037
10%	-0.818618	-0.824686	-0.856301	-0.806868	-0.855829
11%	-0.787823	-0.794837	-0.834615	-0.771993	-0.827903
12%	-0.756305	-0.765638	-0.811573	-0.738513	-0.813643
13%	-0.726605	-0.735462	-0.784663	-0.695729	-0.765959
14%	-0.695377	-0.706243	-0.760018	-0.659897	-0.756154
15%	-0.665054	-0.677629	-0.737292	-0.631092	-0.741744
20%	-0.630705	-0.643865	-0.708544	-0.597312	-0.759030
25%	-0.597631	-0.612905	-0.686275	-0.548669	-0.702058
30%	-0.565641	-0.584952	-0.666811	-0.493476	-0.750499
35%	-0.536420	-0.560730	-0.652261	-0.458648	-0.702436
40%	-0.517770	-0.547288	-0.642082	-0.446062	-0.695310
45%	-0.509191	-0.547264	-0.637989	-0.452451	-0.582652
50%	-0.512050	-0.558067	-0.644999	-0.469800	-0.403287

Table 9: Overall Data – Method II



Graph 1: Overall Data – Method I



Graph 2: Overall Data – Method II

APPENDIX B.

Project Code

```
#Chris Nuhn & Doc Sessions
#6:00 PM 02/16/2010
#research algorithm using Zipf's Powerlaw v4.1
```

```
#####
#          ARGS          #
#####
# 1. String    File      #
# 2. Boolean   Create.xls #
# 3. Boolean   Print     #
#####

if ($#ARGV >= 2) {$print = $ARGV[2];}
if ($#ARGV >= 1) {$xls = $ARGV[1];}
if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
if ($#ARGV < 0) {$error = "file not given\n";}

($first, $extension) = split /\./, $file;
open( OUT, ">$first"."research.txt");

foreach("", .01, .02, .03, .04, .05, .06, .07, .08, .09, .1, .11, .12, .13, .14, .15, .20, .25, .30, .3
5, .40, .45, .50)
{
    open (IN, $_.$file) or die $error;
    $rank = 0;
    foreach (<IN>)
    {
        chomp $_;
        next unless $_ =~ m/\t*(\d+)\t*/;
        $y[$rank] = $1;
        $rank++;
    }

    @y = sort {$b <=> $a} @y;

    $rank = 0;
    foreach (@y)
    {
        $logy[$rank] = log10($_);
        $rank++;
    }

    foreach (1..($#y+1))
    {
        $x[$_ - 1] = $_;
        $logx[$_ - 1] = log10($_);
    }

    print OUT (correlation(\@logx, \@logy, $print) . "\t" . slope(\@logx,
\@logy, $print) . "\n");
}

#####
#          SUBROUTINES    #
#####
sub mean
{
    my ($ref_) = @_;
    my $size = @{$ref_};
    my $mean = 0;
    foreach (@{$ref_})
    {
        $mean += $_;
    }
    return ($mean / $size);
}
```

```

}

sub toxls()
{
    ($first, $extension) = split /\./, $file;
    open( OUT, ">$first"."research.txt");
    foreach (@re)
    {
        print OUT "$_\n";
    }
}

sub sd
{
    my ($ref_) = @_;
    my $size = @{$ref_};
    my $mean = mean($ref_);
    my $sqtotal = 0;
    foreach (@{$ref_})
    {
        $sqtotal += sqr($_ - $mean);
    }
    return sqrt($sqtotal / $size);
}

sub correlation
{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sdx = sd($ref_x);
    my $sdy = sd($ref_y);
    my $xbar = mean($ref_x);
    my $ybar = mean($ref_y);
    my $sum = 0;
    foreach (0..($n - 1))
    {
        $sum += ((@{$ref_x}[$_] - $xbar)*(@{$ref_y}[$_] - $ybar));
    }
    my $corr = ($sum / (($n) * $sdx * $sdy));
    if($print) {print "Correlation:\nr: $corr\n";}
    return $corr;
}

sub sqr
{
    return (@_[0] * @_[0]);
}

sub log10
{
    my ($num) = @_;
    if($num != 0){return (log($num) / log(10))};
}

sub slope
{
    my ($ref_x, $ref_y, $print) = @_;
    my $n = @{$ref_x};
    my $sx = 0;
    my $sy = 0;
    my $sxy = 0;
    my $sxx = 0;
    my $syy = 0;
    foreach(0..(@{$ref_x} - 1))
    {

```



```

    $sx += @{$ref_x}[$_];
    $sy += @{$ref_y}[$_];
    $sxy += (@{$ref_x}[$_] * @{$ref_y}[$_]);
    $sxx += (@{$ref_x}[$_] * @{$ref_x}[$_]);
    $syy += (@{$ref_y}[$_] * @{$ref_y}[$_]);
}
my $m = (($n * $sxy) - ($sx * $sy)) / (($n * $sxx) - sqr($sx));
if($print) {print "\nSlope: $m"};
return ($m);
}

```

```

if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
if ($#ARGV < 0) {$error = "file not given\n";}

open (IN, $file) or die $error;

$rank = 0;
foreach (<IN>)
{
    chomp $_;
    next unless $_ =~ m/\t*(\d+)\t*/;
    $y[$rank] = $_;
    $rank++;
}

@y = sort {$b <=> $a} @y;

foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.
40,.45,.50)
{
    $amount = int($rank * $_);
    $topvalue = int($y[0] * 1.1);

    for($i = 0; $i < $amount; $i++)
    {
        $new[$i] = rand($topvalue);
    }

    foreach (@new)
    {
        $y[rand($rank)] = int($_);
    }

    open( OUT, ">$_."$file");
    foreach (@y)
    {
        print OUT $_, "\n";
    }
}

```

```

if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
if ($#ARGV < 0) {$error = "file not given\n";}

open (IN, $file) or die $error;

$rank = 0;
foreach (<IN>)
{
    chomp $_;
    next unless $_ =~ m/\t*(\d+)\t*/;
    $y[$rank] = $_;
    $rank++;
}

@y = sort {$b <=> $a} @y;

foreach(@y)
{
    $mean += $_;
}

$mean = $mean / $rank;
$topvalue = int(rand(($y[0] * 1.1)));

foreach(.01,.02,.03,.04,.05,.06,.07,.08,.09,.1,.11,.12,.13,.14,.15,.20,.25,.30,.35,.40,.45,.50)
{
    $amount = int($rank * $_);
    for($i = 0; $i < $amount; $i++)
    {
        $new[$i] = rand($topvalue);
    }
    foreach (@new)
    {
        $y[rand($rank)] = int($_);
    }
    open( OUT, ">$_"."pt2"."$file");
    foreach (@y)
    {
        print OUT $_, "\n";
    }
}

```

APPENDIX C.
2010 ICIQ
Research Paper

A DISCUSSION OF METHODS FOR THE DETECTION OF ERRORS IN A POWER LAW DISTRIBUTION

(Research-in-Progress)

Abstract: To detect erroneous data points within a set, one must first understand how the data is usually distributed. Often in our analysis of data, we assume a data set to be normal (that is to follow a normal distribution or be normally distributed). This often is not the case, however, particularly when dealing with human-centric events such as web site hits, word usage, or author citations. In this paper, we examine the power law distribution and the types of data that should be tested for such a distribution. We present test methods and results from the testing of the robustness of a power law distribution under conditions of randomly generated errors in the data set, and discuss methods for error detection in data sets with this type of distribution. We conclude with a discussion of future research.

Key Words: Data Quality, Information Quality, IQ Concepts, Metrics, Measures, and Models

INTRODUCTION

One often begins an analysis of a set of data by plotting the variables as a graph, and determining the mean, median and variance of the data. If the distribution is found to be normal, we unlock a variety of useful functions for dealing with this data. Normal distributions are the most widely used model for random variables with a continuous distribution and often it is a “mathematical convenience to assume that the distribution from which a random sample is drawn is a normal distribution [2]”. While other distributions are also popular and widespread in physical science and human events, often we first consider a normal distribution as a fit for the data set. As we will see in our subsequent analysis, this can obscure the true distribution of the data and therefore meaningful further analysis of outliers or variances in the data set.

As an illustration, we take the data in Table 1 below.

Number	Count	Number	Count
1	214	28	1145
2	227	29	946
3	233	30	840
4	261	31	645
5	277	32	553
6	288	33	544
7	297	34	488
8	303	35	486
9	309	36	414
10	352	37	403
11	366	38	381
12	368	39	375
13	374	40	353
14	403	41	349
15	416	42	340
16	516	43	300
17	525	44	296
18	531	45	286
19	584	46	262
20	663	47	251
21	846	48	239
22	934	49	224
23	1101	50	223
24	2268	51	218
25	2006	52	217
26	8024	53	203
27	5184	54	201

Table 1: Data Counts

Upon initial graphing of this data we see a distribution that appears normal, see Figure 1.

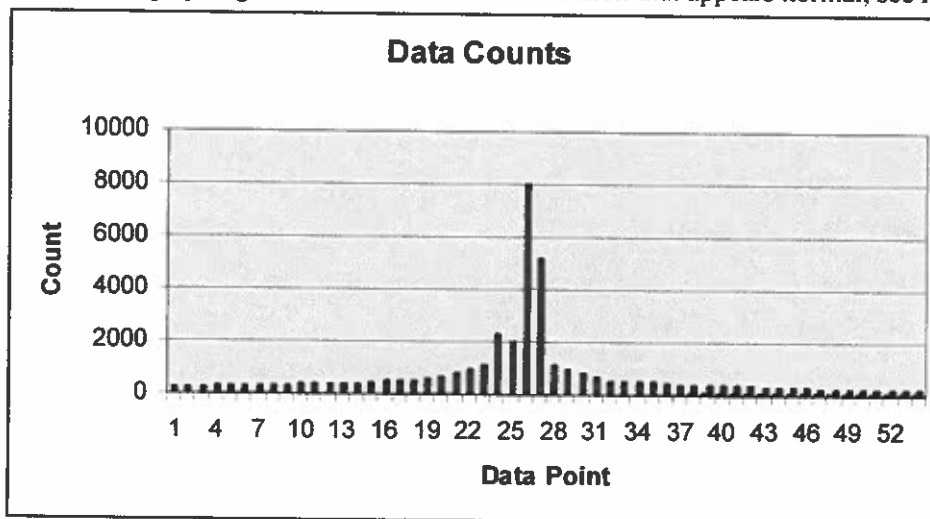


Figure 1: Initial Graphing of data count

Upon examination of mean, median, and mode however we find that the three are close but are not equal.

mean	median	mode
723	371	403

Undaunted we consider the high center point – (#26 – 8024) a probable outlier and the numbers become closer to a normal distribution, as does the graph.

mean	median	mode
497	367	403

Because it is simply more convenient, we then consider the distribution normal, determine the standard deviation and consider all points outside of three standard deviations outliers. In either case above (dropping or keeping data point #26 in our calculation), we can consider the highest value as an outlier ¹.

All of this is nonsense and an example of poor statistical analysis, because as we will see in the discussion below, this set of data is actually linguistic and follows a Zipfian distribution. But for the sake of discussion and illustration, let us look at what would happen if we misclassify the data as normal and have an error occur in the data set. If we erroneously increase data point #21 by a factor of 2 it becomes 1692 and in either scenario this falls within an acceptable range for this distribution and would not be flagged as an outlier.

As we have already discussed, however, this data actually represents word counts inputted to ALICE [11], in the authors' opinion a fantastic Chat Bot developed by Dr. Richard S. Wallace of the ALICE Artificial Intelligence foundation². By understanding that the data is linguistic in nature, we would naturally experiment with a Zipfian or power law distribution instead. Ranking and then plotting the data on a log-log graph we discover the power law distribution of the data as shown in Table 2 and Figure 2.

¹ In the first case $\sigma = 1261$, $3\sigma = 3782$ and in the second case $\sigma = 405$, $3\sigma = 1215$.

² <http://alicebot.blogspot.com/>

Rank	Count	Rank	Count
1	8024	28	368
2	5184	29	366
3	2268	30	353
4	2006	31	352
5	1145	32	349
6	1101	33	340
7	946	34	309
8	934	35	303
9	846	36	300
10	840	37	297
11	663	38	296
12	645	39	288
13	584	40	286
14	553	41	277
15	544	42	262
16	531	43	261
17	525	44	251
18	516	45	239
19	488	46	233
20	486	47	227
21	416	48	224
22	414	49	223
23	403	50	218
24	403	51	217
25	381	52	214
26	375	53	203
27	374	54	201

Table 2: Ranked Data and Count

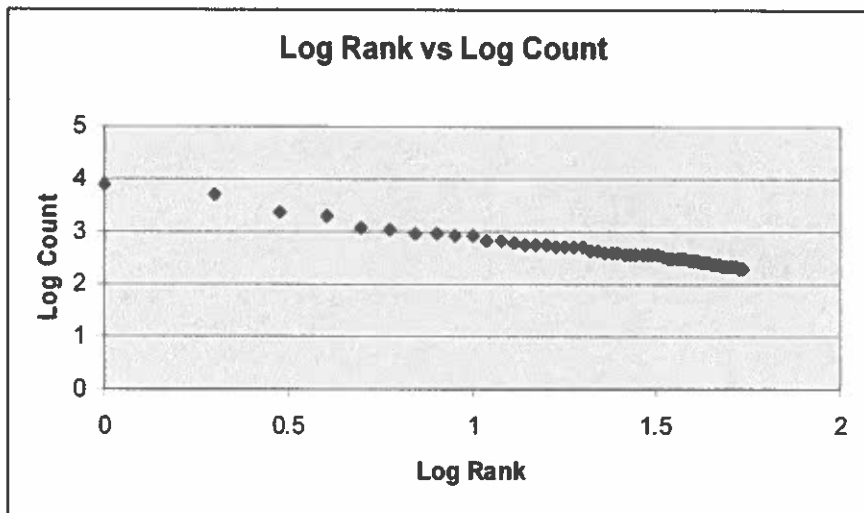


Figure 2: Distribution of data

In this plotting and understanding of the data, if data point #21 (count 846) were to suddenly double, this

would easily be highlighted in the graph and brought to the attention of the data analyst.

Note: This was an illustrative example of the errors that may occur in assuming a data set is normally distributed and does not represent how the ALICE AI Foundation used or analyzed the data above.

While determination of outliers within a normal distribution is commonplace and well represented in the literature, determination of outliers (perhaps errors in the data) in power law distributions is less popular in the literature. In our research, we will attempt to utilize the power law distribution to uncover data inaccuracies in experimental data sets that follow a power law distribution. Our goal is to uncover inaccuracies that might not be easily recognized when data is plotted as a normal distribution. As we present here, during this phase of our research we test the robustness of a power law distribution under randomly generated errors in the set and explore methods for the determination of inaccurate sets.

REVIEW OF LITERATURE

A normal distribution is a special type of standard distribution which follows a bell-shaped curve pattern over a set of data. This bell-shaped curve has several key properties. The distribution is always symmetric, it is concentrated in the center and decreases rapidly on either side, meaning that the data has less frequent near extreme values. The area under the curve has a direct correlation to the probability that that value has of occurring. A large area implies a large probability and a small area implies a small probability [9].

Normal distributions follow the form:

$$F(x) = \frac{e^{\frac{-(x-\mu)^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}}, \text{ where } x \text{ is the variant with mean } \mu \text{ and variance } \sigma^2.$$

Data that follow a normal distribution obey the central limit theorem, which states that for any distribution with mean μ and variance σ^2 , the distribution approaches normal as the sample size continues to increase.

After determining that a function is normal, outliers can then be calculated using Grubb's Test [4].

According to this test, any data point located beyond $(Y - Y_{\min})/s$ or $(Y - Y_{\max})/s$, where Y is the sample mean and s is the standard deviation, is considered an outlier.

While the normal distribution is often assumed, other distributions do exist, including the power law distribution. A power law distribution is a mathematical relationship between two quantities where the number or frequency of an event varies as a power of some attribute of that object. The law normally appears in the form:

$F(x) = ax^k + o(x^k)$, where a and k are constants, $o(x^k)$ is an asymptotically small function of x^k , and k is a scaling exponent [7].

Pareto is the most famous for using this law with his "80-20" theory, which states that 20% of the population holds 80% of the wealth. The graph of this power rank function looks close to exponential, although reversed. Showing that a small percentage contains most of the wealth and as the percentage increases, the distribution falls drastically until it bottoms out along the x-axis. [5].

Zipf's Law is another special type of power law distribution, where the constants are zero. Harvard linguist George Kingsley Zipf observed that if values are given ranks in descending order, that the values are inversely proportional to their ranks compared to the highest rank. That is, the rank two value occurs $\frac{1}{2}$ as much as rank one, while rank three occurs $\frac{1}{3}$ as often.

According to Zipf's law, when the log of the rank is plotted against the log of the values, a line is formed. This is very helpful when determining if a distribution does indeed follow Zipf's law. To determine this, the correlation coefficient is calculated using the logarithmic values of the ranks and data. This

coefficient is a number ranging from $[-1,1]$ where the closer to one, or negative one, the value is, the closer it is to a line. While originally used with linguistic data sets, Zipf's law has been influential in many fields, including computer music [3].

Determining that a data set displays a power law distribution is a challenging research question. The authors shall summarize here pivotal work in this field by Clauset et al [1]. In their research, Clauset et al summarize several data sets that are assumed to have power law distributions, among these count of word use from Moby Dick, book sales, papers authored, and population of cities. They then summarize the most popular method of categorizing a data set as having a power law distribution, which we explained above – plotting the log of the count by the log of the rank of the values and determining through linear regression if the values create a line. There are problems with using this naive method for determining a power law distribution as explained in Clauset et al. Their research goes further and the team creates its own methods for determining a power rank distribution. For a thorough summary of the power law and Clauset et al's work we encourage the reader to view the paper in its entirety. While we did not use their particular methods and algorithms, instead maintaining the usual method of plotting a log-log graph and determining the slope and correlation coefficient, we did rely on their research to determine which data sets should be considered to have follow a power law distribution.

METHODS FOR THE DETECTION OF INACCURATE DATA SETS

Assuming a power law distribution has been determined, we wish to have methods for the detection of errors within the set (or errors in new data points coming into the set). We present and test three methods for determining inaccuracies in this type of distribution:

1. Value of the slope of the line created by plotting the $\log(\text{rank})$ vs $\log(\text{count})$ of the data variables.
2. Change in the value of the slope of the line created by plotting the $\log(\text{rank})$ vs $\log(\text{count})$ of the data variables.
3. Change in the correlation coefficient values (how closely the data fit a line).

In this initial research we chose to analyze the slope of the resulting log-log line, change in the slope from a gold standard data set, and to analyze any changes in the correlation coefficient (how closely the data fit a line). In order to test these methods and determine how data sets with a power law distribution handle errors in the data, we create the following test data sets and perform our tests.

CREATION OF TEST DATA SETS

In order to test the robustness of data sets conforming to a power law distribution under situations of inaccurate data, we created test sets with randomly corrupted data in differing percentages. For instance, we take our initial data set, which follows a power law distribution, and label it the Gold Standard data set. We then use our random number generator to produce erroneous data in differing percentages, 1%, 3%, 15%. etc. Finally, we test the data to determine the resulting slope and correlation coefficients.

In order to create random erroneous data, we corrupted the data using two methods. In the first method, we used the Perl random number generator with a range from 0 to 10 percent greater than the highest value in the particular data set. This appeared to skew the random numbers higher than would be likely in erroneous data, so we also created a second method where we took the randomly generated value between 0 and 10 percent above the highest value, and used that as the top value to randomly generate a second number. Results from both types of randomly generated data errors followed a similar pattern.

See our Perl script in Appendix B.

Our Gold Standard data sets were chosen based on their conformity to a power law distribution as outlined in [1]. We chose sets with p -values greater than 0.10 corresponding to a statistically significant power-law fit. For our research we chose the following data sets and would like to thank Clauset et al for providing links and resources for these and other possible power law distribution data sets on their website:

A) Word Count – this is a count of the frequency of unique words from the novel Moby Dick by Herman Melville [5].

B) Solar Flares – count of the peak intensity of solar flares between 1980-1989 [5].

C) Names – the frequency of the occurrence of family surnames in the US 1990 census [10].

D) Citations – Author citations in June 1997 by authors published in 1981 [6].

E) Religious Affiliation – religious affiliation in the US as reported by the website <http://www.adherents.com>.

Each of the above data sets was considered the Gold Standard set, and was corrupted in different percentages by the two random number generation methods as described above.

TEST RESULTS

Our test results are presented below in chart and graphical form. The results for change in slope were not promising and are therefore shown only in Appendix A. Also, due to space limitations, in Appendix A we show full results for only one of the two random error generation methods because the results were similar. The full results can be obtained by emailing the authors.

Percent Corrupt	R - squared value Random Method I				
	Word Count	Solar Flares	Family Name	Citations	Religious
Gold Standard - 0%	-0.988579	-0.992570	-0.994842	-0.987608	-0.939689
1%	-0.977787	-0.977897	-0.997266	-0.992400	-0.939689
2%	-0.958271	-0.960423	-0.993554	-0.990630	-0.939689
3%	-0.946614	-0.948668	-0.987005	-0.981541	-0.939689
4%	-0.937757	-0.939553	-0.975075	-0.966808	-0.939689
5%	-0.926703	-0.928075	-0.962358	-0.945712	-0.916498
6%	-0.911636	-0.913613	-0.945028	-0.925201	-0.912175
7%	-0.892204	-0.895342	-0.926630	-0.902501	-0.905218
8%	-0.870329	-0.872613	-0.903462	-0.873514	-0.884737
9%	-0.857065	-0.847017	-0.879958	-0.839687	-0.884299
10%	-0.837566	-0.819033	-0.853686	-0.806561	-0.872218
11%	-0.814163	-0.790224	-0.832537	-0.766355	-0.809960
12%	-0.788848	-0.760147	-0.810317	-0.726180	-0.761718
13%	-0.762256	-0.729522	-0.785785	-0.692132	-0.751060
14%	-0.733374	-0.700011	-0.759144	-0.660768	-0.740559
15%	-0.706243	-0.670777	-0.738759	-0.621011	-0.745369
20%	-0.673616	-0.641851	-0.714443	-0.587464	-0.774571
25%	-0.638245	-0.615828	-0.689070	-0.548562	-0.712225
30%	-0.606023	-0.587774	-0.663526	-0.509061	-0.714426
35%	-0.612337	-0.567309	-0.638208	-0.480668	-0.767803
40%	-0.614250	-0.552448	-0.624336	-0.467117	-0.753158
45%	-0.636806	-0.551619	-0.621493	-0.466787	-0.776767
50%	-0.667539	-0.559030	-0.634528	-0.502952	-0.712290

Table 3: Overall results Correlation Coefficient Change (r-squared value) for errors generated with random generation method I

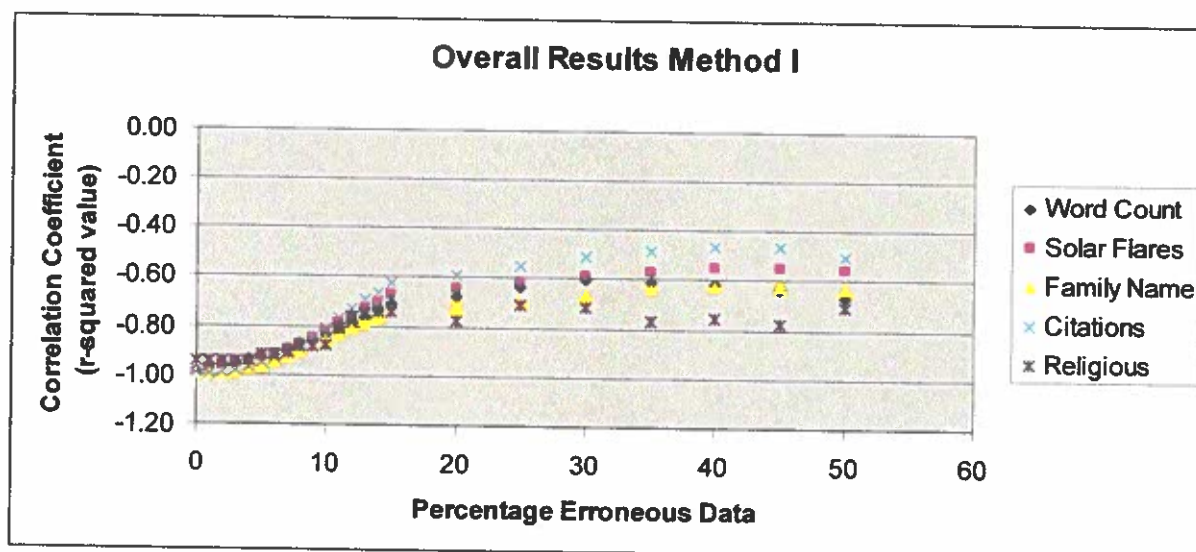


Figure 3: Overall Results Method I

Percent Corrupt	R - squared value Random Method II				
	Word Count	Solar Flares	Family Name	Citations	Religious
Gold Standard - 0%					
1%	-0.988579	-0.992570	-0.994842	-0.987608	-0.939689
2%	-0.978572	-0.981384	-0.995643	-0.992367	-0.939689
3%	-0.961307	-0.966405	-0.993760	-0.990391	-0.939689
4%	-0.950432	-0.954958	-0.986636	-0.981653	-0.939689
5%	-0.941061	-0.944805	-0.975702	-0.966876	-0.939689
6%	-0.929961	-0.933324	-0.961673	-0.947337	-0.925849
7%	-0.915121	-0.918234	-0.945889	-0.922518	-0.905163
8%	-0.896919	-0.899130	-0.927446	-0.899288	-0.893834
9%	-0.873743	-0.876048	-0.905224	-0.869428	-0.869612
10%	-0.847091	-0.852133	-0.881373	-0.834736	-0.856037
11%	-0.818618	-0.824686	-0.856301	-0.806868	-0.855829
12%	-0.787823	-0.794837	-0.834615	-0.771993	-0.827903
13%	-0.756305	-0.765638	-0.811573	-0.738513	-0.813643
14%	-0.726605	-0.735462	-0.784663	-0.695729	-0.765959
15%	-0.695377	-0.706243	-0.760018	-0.659897	-0.756154
20%	-0.665054	-0.677629	-0.737292	-0.631092	-0.741744
25%	-0.630705	-0.643865	-0.708544	-0.597312	-0.759030
30%	-0.597631	-0.612905	-0.686275	-0.548669	-0.702058
35%	-0.565641	-0.584952	-0.666811	-0.493476	-0.750499
40%	-0.536420	-0.560730	-0.652261	-0.458648	-0.702436
45%	-0.517770	-0.547288	-0.642082	-0.446062	-0.695310
50%	-0.509191	-0.547264	-0.637989	-0.452451	-0.582652
	-0.512050	-0.558067	-0.644999	-0.469800	-0.403287

Table 4: Overall results Correlation Coefficient Change (r-squared value) for errors generated with random generation method II

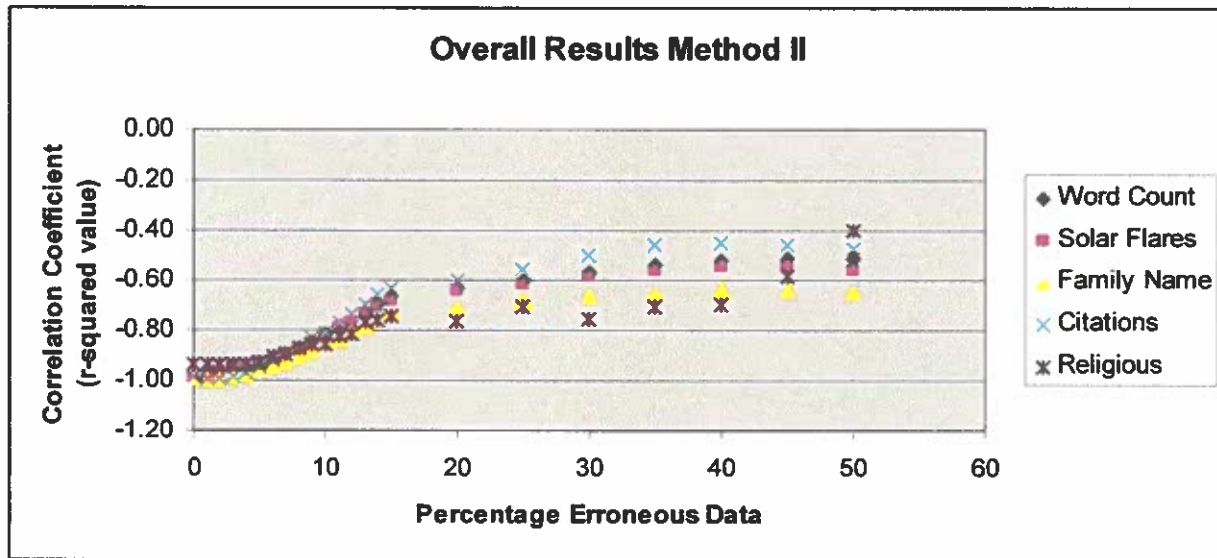


Figure 4: Overall Results Method II

DISCUSSION OF RESULTS

There are many aspects of our test results to comment on. First, it does not appear to significantly change the resulting distribution of slope values or correlation coefficients regardless of which of the two random error generating methods were used. Second, it appears that Methods I and II, slope and change in slope respectively, do not yield quantifiable results. For example, looking at the results in Appendix A, Table 5, one notices that while the slope does change at a predictable rate based on percentage of erroneous data, at 45% inaccurate data the slope itself is very close to the Gold Standard slope. Difference between slope and gold standard slope also do not contain predictable measurements that are useful for our purposes.

The third method, change in correlation coefficient, or r-squared value, however does yield predictable and useful measurements for determining the level of inaccuracies in the data set. We can see from Tables 3 and 4, and Figures 3 and 4, that the correlation coefficient gets farther away from a line (farther from -1 or 1) as the amount of inaccurate data increases. Furthermore, in all but data set E the correlation coefficient values are even similar among the various data sets. These are promising results for many reasons. The slope of the line does change as the amount of inaccurate data increases, however one need not know the original slope of the line to determine the amount of inaccuracy. Instead, if the data is hypothesized to be a power law distribution, it can be plotted and fitted to a line regardless of the slope and an estimate of the level of inaccuracies within the data set can be made.

The gradual nature of the change in the correlation coefficient also suggests that the change could be fitted to a defined correlation between level of inaccuracy and coefficient value so that estimations of the quality of the set could be made. It is too early in our research to strictly define this, but it is interesting that this type of data appears to follow a steady decline as opposed to other sets that have a severe deterioration of distribution with even small amounts of inaccuracies as we have seen in other work [8]. In most instances the data sets remain within a 0.8 r-squared value until around 12% inaccurate data. This is very encouraging for the creation of a robust inaccuracy estimator.

CONCLUSIONS AND FUTURE WORK

This research is still in its early phases. Ultimately the authors would like to test approximately twenty known power law data sets and determine if a definite correlation between r -squared value and level of inaccuracy does indeed exist, and if so to define this. There is also a fourth method that may be promising but that we have not tested that would allow us to determine if there as been an error in one particular data count. If only one value has a perceived error we could account for this by determining if its rank or count changes dramatically over the course of some time period. If there is a dramatic increase this could be flagged and analyzed further. Finally, use of the new methods of Clauset et al [1] and the creation of methods for determining errors using their algorithms could be investigated. Overall, the authors are encouraged by these initial results and will pursue this avenue of research further as many data sets may conform to a power law distribution.

REFERENCES

- [1] Clauset, A, C.R. Shalizi, and M.E.J. Newman. (2009) Power-law distributions in empirical data. *SIAM Review* 51, 661-703.
- [2] DeGroot, M. H. and Mark Schervish. (2002). Probability and Statistics. 3rd Edition. Addison Wesley. Boston.
- [3] Manaris, B. J. Romero, P. Machado, D. Krehbiel, T. Hirzel, W. Pharr, and R. Davis. (2005). "Zipf's Law, Music Classification, and Aesthetics". *Computer Music Journal*. 29 (1). 55-69.
- [4] National Institute of Standards and Technology. Engineering Statistics Handbook. Section 1.5.3.17.
- [5] Newman, M.E.J. (2005). "Power laws, Pareto distributions and Zipf's law." *Contemporary Physics* 46, 323.
- [6] Redner, S. (1998). "How Popular is Your Paper? An Empirical Study of the Citation Distribution." *European Physical Journal B* 4, 131 .
- [7] Reed, W. (2001) The Pareto, Zipf and other power laws. *Economics Letters*, Volume 74, Issue 1.
- [8] Sessions, V. and M. Valtorta. (2009). "Towards a Method for Data Accuracy Assessment Utilizing a Bayesian Network Learning Algorithm." *Journal of Data and Information Quality (JDIQ)*. 1 (3).
- [9] Simon, S. (2008) What is a normal distribution? http://www.cmh.edu/stats/definitions/norm_dist.htm
- [10] US Census Bureau, 1990 US Census, Summary Tape File 1 (STF1) and Summary Tape File 3 (STF3).
- [11] Wallace, R. "Zipf's Law", <http://www.alicebot.org/articles/wallace/zipf.html>

Appendix A Individual Results

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-1.113275	-0.988579	0
1%	-1.330470	-0.977787	0.217195
2%	-1.658394	-0.958271	0.545119
3%	-2.025777	-0.946614	0.912502
4%	-2.383607	-0.937757	1.270332
5%	-2.706081	-0.926703	1.592806
6%	-2.969563	-0.911636	1.856288
7%	-3.157777	-0.892204	2.044502
8%	-3.266302	-0.870329	2.153027
9%	-3.252679	-0.857065	2.139403
10%	-3.192771	-0.837566	2.079495
11%	-3.085507	-0.814163	1.972231
12%	-2.947045	-0.788848	1.833770
13%	-2.783619	-0.762256	1.670344
14%	-2.598766	-0.733374	1.485491
15%	-2.410874	-0.706243	1.297599
20%	-2.173447	-0.673616	1.060172
25%	-1.912633	-0.638245	0.799358
30%	-1.661293	-0.606023	0.548018
35%	-1.484651	-0.612337	0.371376
40%	-1.298752	-0.614250	0.185476
45%	-1.184591	-0.636806	0.071315
50%	-1.091816	-0.667539	-0.021460

Table 5: Data Set A Tabular Results

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-1.069828	-0.992570	0.000000
1%	-1.239464	-0.977897	-0.169636
2%	-1.497125	-0.960423	-0.427297
3%	-1.796500	-0.948668	-0.726672
4%	-2.099655	-0.939553	-1.029827
5%	-2.367906	-0.928075	-1.298078
6%	-2.580713	-0.913613	-1.510885
7%	-2.732337	-0.895342	-1.662509
8%	-2.823911	-0.872613	-1.754083
9%	-2.853433	-0.847017	-1.783605
10%	-2.825839	-0.819033	-1.756011
11%	-2.751598	-0.790224	-1.681770
12%	-2.636783	-0.760147	-1.566955
13%	-2.497380	-0.729522	-1.427552
14%	-2.340690	-0.700011	-1.270862
15%	-2.172721	-0.670777	-1.102893
20%	-1.955555	-0.641851	-0.885727
25%	-1.735631	-0.615828	-0.665803
30%	-1.514631	-0.587774	-0.444803
35%	-1.299938	-0.567309	-0.230110
40%	-1.106034	-0.552448	-0.036206
45%	-0.964522	-0.551619	0.105306
50%	-0.863613	-0.559030	0.206215

Table 6: Data Set B, Tabular Results

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-0.852299	-0.994842	0.000000
1%	-0.920012	-0.997266	-0.067713
2%	-1.042925	-0.993554	-0.190626
3%	-1.168022	-0.987005	-0.315723
4%	-1.315955	-0.975075	-0.463656
5%	-1.447085	-0.962358	-0.594786
6%	-1.564527	-0.945028	-0.712228
7%	-1.640417	-0.926630	-0.788118
8%	-1.690200	-0.903462	-0.837901
9%	-1.700633	-0.879958	-0.848334
10%	-1.691585	-0.853686	-0.839286
11%	-1.655821	-0.832537	-0.803522
12%	-1.595099	-0.810317	-0.742800
13%	-1.524150	-0.785785	-0.671851
14%	-1.446234	-0.759144	-0.593935
15%	-1.368551	-0.738759	-0.516252
20%	-1.267139	-0.714443	-0.414840
25%	-1.151323	-0.689070	-0.299024
30%	-1.045189	-0.663526	-0.192890
35%	-0.927194	-0.638208	-0.074895
40%	-0.819479	-0.624336	0.032820
45%	-0.755504	-0.621493	0.096795
50%	-0.712343	-0.634528	0.139956

Table 7: Data Set C, Tabular Results

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-2.497718	-0.987608	0.000000
1%	-2.678983	-0.992400	-0.181265
2%	-2.936413	-0.990630	-0.438695
3%	-3.289699	-0.981541	-0.791981
4%	-3.640914	-0.966808	-1.143196
5%	-3.961571	-0.945712	-1.463853
6%	-4.167570	-0.925201	-1.669852
7%	-4.306592	-0.902501	-1.808874
8%	-4.379137	-0.873514	-1.881419
9%	-4.370368	-0.839687	-1.872650
10%	-4.260689	-0.806561	-1.762971
11%	-4.073511	-0.766355	-1.575793
12%	-3.841621	-0.726180	-1.343903
13%	-3.599693	-0.692132	-1.101975
14%	-3.311202	-0.660768	-0.813484
15%	-2.988006	-0.621011	-0.490288
20%	-2.681185	-0.587464	-0.183467
25%	-2.341687	-0.548562	0.156031
30%	-1.927895	-0.509061	0.569823
35%	-1.602143	-0.480668	0.895575
40%	-1.346614	-0.467117	1.151104
45%	-1.111651	-0.466787	1.386067
50%	-1.023600	-0.502952	1.474118

Table 8: Data Set D, Tabular Results

Percent Corrupt	Slope	r squared value	Distance from Gold Standard Slope
Gold Standard (0%)	-3.170003905	-0.939689	0.000000
1%	-3.170003905	-0.939689	0.000000
2%	-3.170003905	-0.939689	0.000000
3%	-3.170003905	-0.939689	0.000000
4%	-3.170003905	-0.939689	0.000000
5%	-3.196891387	-0.916498	-0.026887
6%	-3.166925523	-0.912175	0.003078
7%	-3.187790214	-0.905218	-0.017786
8%	-3.028894425	-0.884737	0.141109
9%	-3.025066111	-0.884299	0.144938
10%	-3.036999758	-0.872218	0.133004
11%	-2.80998711	-0.809960	0.360017
12%	-2.677691993	-0.761718	0.492312
13%	-2.667245636	-0.751060	0.502758
14%	-2.653013085	-0.740559	0.516991
15%	-2.634523768	-0.745369	0.535480
20%	-2.668712705	-0.774571	0.501291
25%	-2.089574955	-0.712225	1.080429
30%	-2.046534647	-0.714426	1.123469
35%	-1.84755318	-0.767803	1.322451
40%	-1.263565195	-0.753158	1.906439
45%	-1.015029791	-0.776767	2.154974
50%	-1.121950978	-0.712290	2.048053

Table 9: Data Set E, Tabular Results

Appendix B

Sample Perl scripts. For more source code contact the authors. This was compiled under Perl 5.10.1.

#Chris Nuhn & Doc Sessions

#6:00 PM 02/16/2010

#Zipf's Powerlaw v3.2

```
#####
#      ARGS      #
#####
# 1. String      File      #
# 2. Boolean      Create.xls  #
# 3. Boolean      Print      #
#####

if ($#ARGV >= 2) {$print = $ARGV[2];}
if ($#ARGV >= 1) {$xls = $ARGV[1];}
if ($#ARGV >= 0) {$file = $ARGV[0]; $error = "$file not found\n";}
if ($#ARGV < 0) {$error = "file not given\n";}

open (IN, $file) or die $error;
$rank = 0;
foreach (<IN>)
{
    chomp $_;
    next unless $_ =~ m/!(\d+)(t*)/;
    $y[$rank] = $1;
    $rank++;
}
}
```

```

@y = sort {$b <=> $a} @y;

$rank = 0;
foreach (@y)
{
    $logy[$rank] = log10($_);
    $rank++;
}

foreach (1..($#y+1))
{
    $x[$_ - 1] = $_;
    $logx[$_ - 1] = log10($_);
}

correlation(\@logx, \@logy, $print);
if($xls) {toxls();}

#####
#      SUBROUTINES      #
#####
sub mean
{
    my ($ref_) = @_;
    my $size = @{$ref_};
    my $mean = 0;
    foreach (@{$ref_})
    {
        $mean += $_;
    }
    return ($mean / $size);
}

sub toxls()
{
    ($first, $extension) = split /\./, $file;
    open( OUT, ">$first"."log.xls");
    foreach (0..$#y)
    {
        print OUT $x[$_], "\t", $y[$_], "\t", $logx[$_], "\t", $logy[$_], "\n";
    }
    print "$first"."log.xls created\n";
}

sub sd
{
    my ($ref_) = @_;
    my $size = @{$ref_};
    my $mean = mean($ref_);
    my $sqtotal = 0;
    foreach (@{$ref_})
    {
        $sqtotal += sqr($_ - $mean);
    }
}

```

```

    }
    return sqrt($sqtotal / $size);
}

sub correlation
{
    my ($ref_x, $ref_y, $print) = @_ ;
    my $n = @{$ref_x};
    my $sdx = sd($ref_x);
    my $sdy = sd($ref_y);
    my $xbar = mean($ref_x);
    my $ybar = mean($ref_y);
    my $sum = 0;
    foreach (0..($n - 1))
    {
        $sum += ((@{$ref_x}[$_] - $xbar)*(@{$ref_y}[$_] - $ybar));
    }
    my $corr = ($sum / (($n) * $sdx * $sdy));
    if($print) {print "Correlation:\nr: $corr\n";}
    return $corr;
}

sub sqr
{
    return (@_[0] * @_[0]);
}

sub log10
{
    my ($num) = @_ ;
    return (log($num) / log(10))
}

```

APPENDIX D.
2010 Spring
Symposium
Presentation



Data Quality Research: Finding Anomalies Using Power Rank Distributions

Valerie Sessions Chris Nuhn
March 18, 2010

1

CHARLESTON SOUTHERN UNIVERSITY
Computer and Information Sciences



Our Work to Date

- Research in blending machine learning algorithms and DQ assessments
- Multiple conference publications on this topic – at the MIT International Conference on Information Quality (ICIQ)
- Journal of Data and Information Quality – December 2009
- Research in improving the quality of LE data sets
- Grants from National Institute of Justice (NIJ) and presentations at NIJ Conferences

3

CHARLESTON SOUTHERN UNIVERSITY
Computer and Information Sciences



Data Quality

- Field blending the fields of computer science and business
- Masters Programs at MIT, Univ. Arkansas, Little Rock
- Define metrics such as accuracy, believability, completeness, timeliness
- Define methods for mapping quality issues
- Cost effective methods for improving DQ in organizations

2

CHARLESTON SOUTHERN UNIVERSITY
Computer and Information Sciences



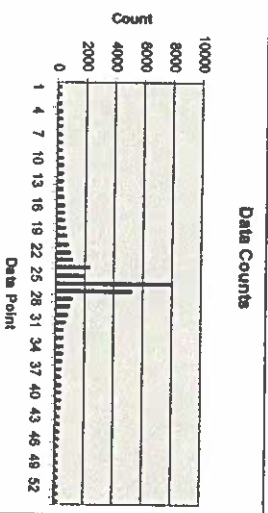
New Work – Zips Law

- Blends the two research facets
- New methods for detecting anomalies that may be outliers in special distributions
- Using LE data sets (specifically criminal activity codes) as test sets
- Senior Project Program for Mr. Chris Nuhn

4

CHARLESTON SOUTHERN UNIVERSITY
Computer and Information Sciences

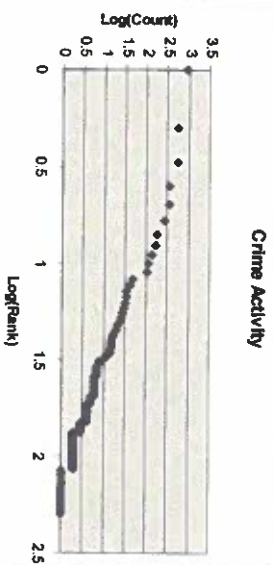
Normal Distribution



Code to Test

```
sub correlation
{
  my ($ref_x, $ref_y, $print) = @_;
  my $sx = @{$ref_x};
  my $sdx = sd($ref_x);
  my $sy = sd($ref_y);
  my $sbar = mean($ref_x);
  my $sum = 0;
  for each (0..($sx - 1))
  {
    $sum += ((@{$ref_x}[$s_] - $sbar)*(@{$ref_y}[$s_] - $sbar));
  }
  my $corr = ($sum / (($sx * $sdx * $sy));
  if ($print) {print "Correlation: ", $corr, "\n";}
  return $corr;
}
```

Power Rank – LE Data Set



Future Plans

- Develop programs to ensure distribution is not normal
- Determine multiple methods for determining anomalies in power rank distributions
- Submit joint paper for ICIQ 2010



Thank You!

Contact Information:

Valerie Sessions

Chris Nuhn
ccckspyder@hotmail.com

CHARLOTTE-MECKLENBURG COLLEGE

Computer and Information Sciences

APPENDIX E.

Senior Defense Presentation

Data Quality and Zipf's Law

Chris Nuhn

Dr. Valerie Sessions

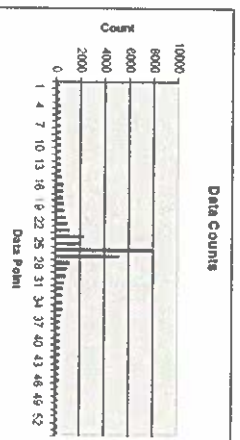
November 29, 2010

Project Purpose

- To explore new methods for finding errors in a set of data
 - Normal Distribution
 - Zipf's Law
 - Outliers

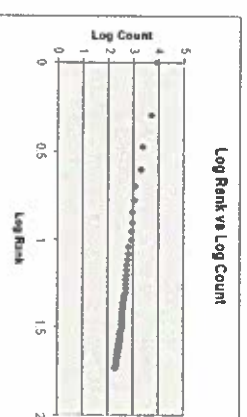
Normal Distributions

- What is a Normal Distribution?
- Why is it so commonly used?



Power-rank and Zipf's Law

- What is Zipf's Law?
- Why did we choose Zipf's Law



Outliers

- Normal Distributions
- Grubbs Test
- Zipf's Law?
- Slope
- Correlation Coefficient

Results

Percent Corrupt	Slope	r-squared value	Distance from Gold Standard Slope
Gold Standard 0%	-1.113275	-0.988379	0
1%	-1.330470	-0.977797	0.217195
2%	-1.539394	-0.96427	0.45119
3%	-1.726777	-0.94864	0.912502
4%	-1.985607	-0.927747	1.20332
5%	-2.309081	-0.926703	1.592805
6%	-2.692463	-0.911636	1.855788
7%	-3.157777	-0.897204	2.044502
8%	-3.266302	-0.870299	2.153027
9%	-3.256579	-0.867065	2.153403
10%	-3.192771	-0.871565	2.073495
11%	-3.006507	-0.811463	1.972231
12%	-2.847045	-0.768849	1.833770
13%	-2.847045	-0.768849	1.833770
14%	-2.598766	-0.723374	1.485481
15%	-2.410874	-0.706243	1.291595
16%	-2.175147	-0.673616	1.060772
17%	-1.919535	-0.618455	0.799358
18%	-1.667293	-0.604023	0.546018
19%	-1.404561	-0.613387	0.371316
20%	-1.1398752	-0.611250	0.185416
40%	-1.16459	-0.648906	0.011315
60%	1.097816	-0.601539	-0.061460

Corrupting Data

- Two Different Methods
- Random Number Generator
- Gold Standard
- 1-15%, 20-50% incrementing by 5

Results Overall

Percent Corrupt	R-squared value	Random Method	Early Name	Chadrons	HiGrowth
Gold Standard 0%	-0.993579	-0.993570	-0.993432	-0.987606	-0.996835
1%	-0.97787	-0.977897	-0.991266	-0.992400	-0.998639
2%	-0.953217	-0.965173	-0.991524	-0.99610	-0.99664
3%	-0.928712	-0.945845	-0.991025	-0.991541	-0.998635
4%	-0.917135	-0.935532	-0.991005	-0.995105	-0.998635
5%	-0.911636	-0.935532	-0.991005	-0.995105	-0.998635
6%	-0.911636	-0.935532	-0.991005	-0.995105	-0.998635
7%	-0.902934	-0.935142	-0.995340	-0.996201	-0.998635
8%	-0.90135	-0.935142	-0.995340	-0.996201	-0.998635
9%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
10%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
11%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
12%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
13%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
14%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
15%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
16%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
17%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
18%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
19%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
20%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
25%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
30%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
35%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
40%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
45%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635
50%	-0.897065	-0.935142	-0.995340	-0.996201	-0.998635

Code Breakdown

```
def correlation
{
  my $ref_x = @{$ref_y, sprintf "%d",
  my $idx = 0;
  my $sidx = 0;
  my $sidx = mean($ref_x);
  my $sidx = mean($ref_x);
  foreach (0..($n - 1))
  {
    $sum += (($ref_x[$sidx] - $sidx) * ($ref_y[$sidx] - $sidx));
  }
  my $score = ($sum / (($n) * $sidx * $sidx));
  if ($score) {print "Correlation: ", $score, "\n";}
  return $score;
}
```

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{(n-1)S_x S_y}$$

Challenges

- Format of .txt file
- Random Number Generator
- Log Bases

Research

Code Breakdown

Problems

Appendix A

Appendix B

Appendix C

Appendix D

Appendix E